

La Tienda - Sistema E-Commerce con Arquitectura de Microservicios

Integrantes

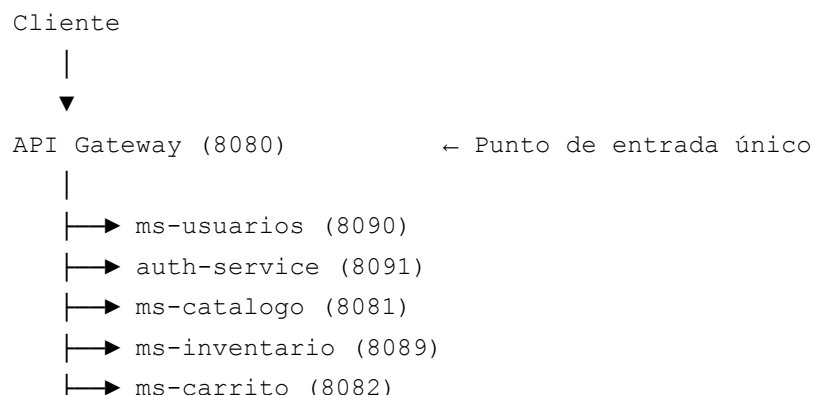
Nombre	Rol en el proyecto
Joaquin Rojas	Infraestructura, autenticación y microservicios de negocio core
Diego Rivera	Microservicios de soporte y gestión de negocio

Descripción del Proyecto

La Tienda es un sistema de e-commerce construido sobre una arquitectura de microservicios utilizando Java 21 y Spring Boot 3.5.14. El sistema distribuye la lógica de negocio en 13 microservicios independientes, cada uno con su propia base de datos MySQL, comunicándose entre sí a través de OpenFeign y registrados en un servidor Eureka para descubrimiento de servicios.

El proyecto resuelve el problema de escalabilidad y mantenibilidad de un e-commerce monolítico, distribuyendo cada responsabilidad de negocio en servicios autónomos que pueden desplegarse, escalarse y mantenerse de forma independiente.

Arquitectura del Sistema



└─> ms-pedidos (8087)
└─> ms-pagos (8086)
└─> ms-envios (8083)
└─> ms-notificaciones (8085)
└─> ms-marketing (8084)
└─> ms-resenas (8088)

Eureka Server (8761)

← Descubrimiento de servicios

Stack Tecnológico

Tecnología	Versión	Uso
Java	21	Lenguaje principal
Spring Boot	3.5.14	Framework base
Spring Cloud	2025.0.2	Infraestructura de microservicios
Netflix Eureka	-	Descubrimiento de servicios
Spring Cloud Gateway	-	API Gateway y enrutamiento
OpenFeign	-	Comunicación entre microservicios
Spring Security	-	Seguridad y autenticación
JWT (jjwt 0.12.3)	0.12.3	Generación y validación de tokens
Spring Data JPA	-	Persistencia de datos
Hibernate	6.6.x	ORM
MySQL	8.0	Base de datos (puerto 3307)
Lombok	-	Reducción de código repetitivo
Maven	-	Gestión de dependencias

Microservicios Implementados

Infraestructura

Microservicio	Puerto	Descripción
eureka-server	8761	Servidor de descubrimiento de servicios
api-gateway	8080	Punto de entrada único al sistema

Seguridad y Usuarios

Microservicio	Puerto	Base de Datos	Descripción
ms-usuarios	8090	ms_usuarios	Registro y gestión de usuarios con roles (ADMIN/CLIENTE)
auth-service	8091	ms_auth	Autenticación con JWT (login y validación de tokens)

Negocio

Microservicio	Puerto	Base de Datos	Descripción
ms-catalogo	8081	ms_catalogo	Gestión de productos. Integra stock via Feign
ms-inventario	8089	ms_inventario	Control de stock por producto
ms-carrito	8082	ms_carrito	Carrito de compras con verificación de stock y precio en tiempo real
ms-pedidos	8087	ms_pedidos	Creación y consulta de pedidos desde carrito confirmado
ms-pagos	8086	ms_pagos	Procesamiento de pagos simulado (80% éxito, 20% fallo)
ms-envios	8083	ms_envios	Gestión de envíos con obtención automática de dirección
ms-notificaciones	8085	ms_notificaciones	Historial de notificaciones por usuario
			Gestión de promociones con

ms-marketing	8084	ms_marketing	descuento por porcentaje
ms-resenas	8088	ms_resenas	Reseñas y puntuaciones de productos (1-5 estrellas)

Funcionalidades Implementadas

Seguridad

- Registro de usuarios con validación de email único y formato correcto
- Autenticación con JWT firmado con algoritmo HS256
- Roles de usuario: ADMIN y CLIENTE
- Validación de tokens con extracción de email y rol

Catálogo e Inventario

- CRUD completo de productos con validaciones
- Gestión de stock independiente por producto
- Endpoint de detalle que combina producto y stock en tiempo real via Feign

Flujo de Compra

- Carrito de compras con verificación de stock y precio en tiempo real
- Estados de carrito: ACTIVO, CONFIRMADO, CANCELADO
- Creación de pedidos desde carrito confirmado
- Procesamiento de pagos simulado
- Gestión de envíos con tracking de estados
- Registro automático de fecha de entrega

Soporte

- Sistema de notificaciones con historial y estados de lectura
- Gestión de promociones con descuentos por porcentaje
- Reseñas de productos con cálculo de promedio

Comunicación entre Servicios (OpenFeign)

- ms-catalogo → ms-inventario (stock en tiempo real)
 - ms-carrito → ms-catalogo (precio) y ms-inventario (stock)
 - ms-pedidos → ms-carrito (datos del carrito)
 - ms-envios → ms-usuarios (dirección de entrega)
 - auth-service → ms-usuarios (verificación de credenciales)
-



Pasos para Ejecutar el Proyecto

Prerrequisitos

- Java 21 instalado
- Maven instalado
- XAMPP con MySQL corriendo en el puerto **3307**
- Las bases de datos se crean automáticamente al arrancar cada servicio

Orden de arranque

Ejecutar `mvn spring-boot:run` (o `mvnw.cmd spring-boot:run` en Windows) desde la carpeta de cada servicio en el siguiente orden:

```
# 1. Servidor de descubrimiento (esperar hasta ver "Started EurekaServerApplicati
cd eureka-server
mvn spring-boot:run
```

```
# 2. Microservicios de negocio (abrir terminal separada para cada uno)
cd ms-inventario && mvn spring-boot:run
cd ms-catalogo && mvn spring-boot:run
cd ms-usuarios && mvn spring-boot:run
cd auth-service && mvn spring-boot:run
cd ms-carrito && mvn spring-boot:run
cd ms-pedidos && mvn spring-boot:run
cd ms-pagos && mvn spring-boot:run
cd ms-envios && mvn spring-boot:run
cd ms-notificaciones && mvn spring-boot:run
cd ms-marketing && mvn spring-boot:run
cd ms-resenas && mvn spring-boot:run
```

```
# 3. Gateway (arrancar al final)
cd api-gateway && mvn spring-boot:run
```

Verificación

1. Abrir `http://localhost:8761` para verificar que todos los servicios estén registrados en Eureka
2. Todas las peticiones se realizan a través del Gateway: `http://localhost:8080`

Configuración de base de datos

Cada servicio tiene su propio `application.properties` con:

```
spring.datasource.url=jdbc:mysql://localhost:3307/nombre_bd?createDatabaseIfNotEx
spring.datasource.username=root
spring.datasource.password=
```

Las bases de datos se crean automáticamente. Si MySQL tiene contraseña configurada, actualizarla en el `application.properties` de cada servicio.

Endpoints Principales

Todos los endpoints se consumen a través del API Gateway en `http://localhost:8080`

Método	Endpoint	Descripción
POST	<code>/api/usuarios</code>	Crear usuario
POST	<code>/auth/login</code>	Login y obtención de JWT
GET	<code>/auth/validate?token=</code>	Validar token JWT
POST	<code>/api/catalogo/productos</code>	Crear producto
GET	<code>/api/catalogo/productos/{id}/detalles</code>	Ver producto con stock
POST	<code>/api/inventario/guardar</code>	Agregar stock
POST	<code>/api/carrito/{usuarioId}/agregar</code>	Agregar producto al carrito
PATCH	<code>/api/carrito/{carritoId}/estado</code>	Cambiar estado del carrito

POST	/api/pedidos/crear?usuarioId=&carritoId=	Crear pedido
POST	/api/pagos/procesar	Procesar pago
POST	/api/envios/crear	Crear envío
PATCH	/api/envios/{envioId}/estado	Actualizar estado del envío
POST	/api/notificaciones/enviar	Enviar notificación
POST	/api/marketing/promociones	Crear promoción
POST	/api/resenas	Crear reseña
GET	/api/resenas/producto/{id}/promedio	Ver promedio de puntuación



Ejemplos de Uso

1. Registrar un usuario

POST http://localhost:8080/api/usuarios
Content-Type: application/json

```
{
  "nombre": "Joaquin Rojas",
  "email": "joaquin@tienda.com",
  "password": "joaquin123",
  "telefono": "912345678",
  "direccion": "Av. Los Leones 123",
  "rol": "ADMIN"
}
```

Respuesta 201 Created:

```
{
  "id": 1,
  "nombre": "Joaquin Rojas",
  "email": "joaquin@tienda.com",
  "rol": "ADMIN"
}
```

2. Login y obtención de JWT

```
POST http://localhost:8080/auth/login
Content-Type: application/json
```

```
{
  "email": "joaquin@tienda.com",
  "password": "joaquin123"
}
```

Respuesta 200 OK:

```
{
  "tipo": "Bearer",
  "token": "eyJhbGciOiJIUzI4NCJ9..."
}
```

3. Validar token JWT

```
GET http://localhost:8080/auth/validate?token=eyJhbGciOiJIUzI4NCJ9...
```

Respuesta 200 OK:

```
{
  "valido": "true",
  "email": "joaquin@tienda.com",
  "rol": "ADMIN"
}
```

4. Crear producto en catálogo

```
POST http://localhost:8080/api/catalogo/productos
Content-Type: application/json
```

```
{
  "nombre": "Laptop Gamer RTX",
  "descripcion": "Laptop de alto rendimiento con GPU dedicada",
  "precio": 1299.99
}
```

5. Agregar stock al inventario

```
POST http://localhost:8080/api/inventario/guardar
```


Content-Type: application/json

```
{
  "productoId": 1,
  "cantidad": 50
}
```

6. Ver producto con stock (comunicación entre servicios via Feign)

GET http://localhost:8080/api/catalogo/productos/1/detalles

Respuesta 200 OK :

```
{
  "id": 1,
  "nombre": "Laptop Gamer RTX",
  "descripcion": "Laptop de alto rendimiento con GPU dedicada",
  "precio": 1299.99,
  "stockDisponible": 50
}
```

7. Agregar producto al carrito

POST http://localhost:8080/api/carrito/1/agregar

Content-Type: application/json

```
{
  "productoId": 1,
  "cantidad": 2
}
```

Respuesta 200 OK :

```
{
  "carritoId": 1,
  "usuarioId": 1,
  "estado": "ACTIVO",
  "items": [
    {
      "itemId": 1,
      "productoId": 1,
      "cantidad": 2,
      "precioUnitario": 1299.99,
    }
  ]
}
```

```
        "subtotal": 2599.98
      }
    ],
    "total": 2599.98
  }
}
```

8. Confirmar carrito

PATCH <http://localhost:8080/api/carrito/1/estado>
Content-Type: application/json

```
{
  "estado": "CONFIRMADO"
}
```

9. Crear pedido desde carrito confirmado

POST <http://localhost:8080/api/pedidos/crear?usuarioId=1&carritoId=1>

Respuesta 201 Created:

```
{
  "pedidoId": 1,
  "usuarioId": 1,
  "estado": "PENDIENTE",
  "total": 2599.98,
  "fechaCreacion": "2026-05-17T19:00:00",
  "items": [...]
}
```

10. Procesar pago (80% éxito, 20% fallo simulado)

POST <http://localhost:8080/api/pagos/procesar>
Content-Type: application/json

```
{
  "pedidoId": 1,
  "usuarioId": 1,
  "monto": 2599.98
}
```

Respuesta 201 Created:

```
{
  "pagoId": 1,
  "estado": "COMPLETADO",
  "detalle": "Pago procesado exitosamente.",
  "fechaPago": "2026-05-17T19:05:00"
}
```

11. Crear envío (obtiene dirección de ms-usuarios via Feign)

POST http://localhost:8080/api/envios/crear
Content-Type: application/json

```
{
  "pedidoId": 1,
  "usuarioId": 1
}
```

Respuesta 201 Created:

```
{
  "envioId": 1,
  "direccionEntrega": "Av. Los Leones 123",
  "estado": "PENDIENTE",
  "fechaCreacion": "2026-05-17T19:10:00",
  "fechaEntrega": null
}
```

12. Actualizar estado del envío

PATCH http://localhost:8080/api/envios/1/estado
Content-Type: application/json

```
{
  "estado": "ENTREGADO"
}
```

Respuesta 200 OK:

```
{
  "envioId": 1,
  "estado": "ENTREGADO",
  "fechaEntrega": "2026-05-17T20:00:00"
}
```

```
}
```

13. Enviar notificación

```
POST http://localhost:8080/api/notificaciones/enviar
Content-Type: application/json
```

```
{
  "usuarioId": "1",
  "tipo": "PAGO_COMPLETADO",
  "mensaje": "Tu pago fue aprobado exitosamente."
}
```

14. Crear reseña de producto

```
POST http://localhost:8080/api/resenas
Content-Type: application/json
```

```
{
  "productoId": 1,
  "usuarioId": 1,
  "puntuacion": 5,
  "comentario": "Excelente producto, muy buena calidad."
}
```

15. Ver promedio de puntuación de un producto

```
GET http://localhost:8080/api/resenas/producto/1/promedio
```

Respuesta 200 OK :

```
{
  "productoId": 1,
  "promedioPuntuacion": 4.5,
  "escala": "1 a 5 estrellas"
}
```

16. Crear promoción

```
POST http://localhost:8080/api/marketing/promociones
Content-Type: application/json
```

```
{
  "nombre": "Black Friday",
  "descripcion": "Descuento especial de temporada",
  "descuentoPorcentaje": 25
}
```

Estructura del Repositorio

```
proyecto-tienda-microservicios/
├─ eureka-server/
├─ api-gateway/
├─ ms-usuarios/
├─ auth-service/
├─ ms-catalogo/
├─ ms-inventario/
├─ ms-carrito/
├─ ms-pedidos/
├─ ms-pagos/
├─ ms-envios/
├─ ms-notificaciones/
├─ ms-marketing/
├─ ms-resenas/
├─ tienda.code-workspace
└─ README.md
```

Cada microservicio sigue la estructura estándar de Spring Boot:

```
ms-nombre/
├─ src/main/java/DJ/TIENDA/ms_nombre/
│   └─ MsNombreApplication.java
│   └─ controller/
│   └─ service/
│   └─ repository/
│   └─ model/
│   └─ dto/
│       └─ client/ (solo si usa Feign)
├─ src/main/resources/
│   └─ application.properties
└─ pom.xml
```